

AFRILANG Stdlib

std/c/mlensemble

Français. Bibliothèque AFRILANG 'std/c/mlensemble' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : enseSum, enseMean, enseMin, enseMax, enseVar, enseStd, enseNorm.

```
'import "std/c/mlensemble"' · 'use mlensemble'
```

- 'enseSum(v list number) → number' - 'enseMean(v list number) → number' - 'enseMin(v list number) → number' - 'enseMax(v list number) → number' - 'enseVar(v list number) → number' - 'enseStd(v list number) → number' - 'enseNorm(v list number) → list number'

English. AFRILANG library 'std/c/mlensemble' (tier complex, category complex). Exports 7 function(s): enseSum, enseMean, enseMin, enseMax, enseVar, enseStd, enseNorm.

```
'import "std/c/mlensemble"' · 'use mlensemble'
```

- 'enseSum(v list number) → number' - 'enseMean(v list number) → number' - 'enseMin(v list number) → number' - 'enseMax(v list number) → number' - 'enseVar(v list number) → number' - 'enseStd(v list number) → number' - 'enseNorm(v list number) → list number'

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/mlensemble' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : enseSum, enseMean, enseMin, enseMax, enseVar, enseStd, enseNorm.

```
'import "std/c/mlensemble"' · 'use mlensemble'
```

```
- `enseSum(v list number) → number`
- `enseMean(v list number) → number`
- `enseMin(v list number) → number`
- `enseMax(v list number) → number`
- `enseVar(v list number) → number`
- `enseStd(v list number) → number`
- `enseNorm(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlensemble"
use mlensemble
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- enseSum(v list number) → number•
enseMean(v list number) → number•
enseMin(v list number) → number•
enseMax(v list number) → number•
enseVar(v list number) → number•
enseStd(v list number) → number•
enseNorm(v list number) → list

4. Exemples d'utilisation

```
import "std/c/mlensemble"
use mlensemble
```

```
create result = enseSum(/* args */)
say result
```

```
create result = enseMean(/* args */)
say result
```

```
create result = enseMin(/* args */)
say result
```

```
create result = enseMax(/* args */)
say result
```

```
create result = enseVar(/* args */)
say result
```

```
create result = enseStd(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/mlensemble"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module mlensemble

```
function enseSum(v list number) returns number
end
```

```
function enseMean(v list number) returns number
end
```

```
function enseMin(v list number) returns number
end
```

```
function enseMax(v list number) returns number
end
```

```
function enseVar(v list number) returns number
end
```

```
function enseStd(v list number) returns number
end
```

```
function enseNorm(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/mlensemble' (tier complex, category complex). Exports 7 function(s): enseSum, enseMean, enseMin, enseMax, enseVar, enseStd, enseNorm.

'import "std/c/mlensemble"' · 'use mlensemble'

```
- `enseSum(v list number) → number`
- `enseMean(v list number) → number`
- `enseMin(v list number) → number`
- `enseMax(v list number) → number`
- `enseVar(v list number) → number`
- `enseStd(v list number) → number`
- `enseNorm(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlensemble"
use mlensemble
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- enseSum(v list number) → number•
enseMean(v list number) → number•
enseMin(v list number) → number•
enseMax(v list number) → number•
enseVar(v list number) → number•
enseStd(v list number) → number•
enseNorm(v list number) → list

4. Usage examples

```
import "std/c/mlensemble"
use mlensemble
```

```
create result = enseSum(/* args */)
say result
```

```
create result = enseMean(/* args */)
say result
```

```
create result = enseMin(/* args */)
say result
```

```
create result = enseMax(/* args */)
say result
```

```
create result = enseVar(/* args */)
say result
```

```
create result = enseStd(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/mlensemble"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module mlensemble

```
function enseSum(v list number) returns number
end
```

```
function enseMean(v list number) returns number
end
```

```
function enseMin(v list number) returns number
end
```

```
function enseMax(v list number) returns number
end
```

```
function enseVar(v list number) returns number
end
```

```
function enseStd(v list number) returns number
end
```

```
function enseNorm(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mlensemble"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/mlensemble/>

Source (extrait)

```
module mlensemble

function enseSum(v list number) returns number
end

function enseMean(v list number) returns number
```

```
end

function enseMin(v list number) returns number
end

function enseMax(v list number) returns number
end

function enseVar(v list number) returns number
end

function enseStd(v list number) returns number
end

function enseNorm(v list number) returns list number
end
end
```